Accelerating Image-based Pest Detection on a Heterogeneous Multicore Microcontroller

Luca Bompani, *Student Member, IEEE*, Luca Crupi, Daniele Palossi, Olmo Baldoni, Davide Brunelli, *Senior Member, IEEE*, Francesco Conti, *Member, IEEE*, Manuele Rusci, *Member, IEEE*, and Luca Benini, *Fellow, IEEE*

***Abstract*—The codling moth pest poses a significant threat to global crop production, with potential losses of up to 80% in apple orchards. Special camera-based sensor nodes are deployed in the field to record and transmit images of trapped insects to monitor the presence of the pest. This article investigates the embedding of computer vision algorithms in the sensor node using a novel state-of-the-art microcontroller unit (MCU), the GreenWaves Technologies' GAP9 system-on-chip, which combines 10 RISC-V general purposes cores with a convolution hardware accelerator. We compare the performance of a lightweight Viola–Jones detector algorithm with a convolutional neural network (CNN), MobileNetV3-SSDLite, trained for the pest detection task. On two datasets that differentiate for the distance between the camera sensor and the pest targets, the CNN generalizes better than the other method and achieves a detection accuracy between 83% and 72%. Thanks to the GAP9's CNN accelerator, the CNN inference task takes only 147 ms to process a 320 × 240 pixel image. Compared to the GAP8 MCU, which only relies on general-purpose cores for processing, we achieved 9.5× faster inference speed. When running on a 1000 mAh battery at 3.7 V, the estimated lifetime is approximately 199 days, processing an image every 30 s. Our study demonstrates that the novel heterogeneous MCU can perform end-to-end CNN inference with an energy consumption of just 4.85 mJ, matching the efficiency of the simpler Viola–Jones algorithm and offering power consumption up to 15× lower than previous methods.**

***Index Terms*—Codling moth, convolutional neural network (CNN), microcontroller, pest detection, smart agriculture, Viola–Jones.**

Received 12 February 2024; revised 20 May 2024 and 20 August 2024; accepted 21 August 2024. Date of publication 25 September 2024; date of current version 10 October 2024. This work was supported in part by the GEMINI "Green Machine Learning for the IoT " national research project and in part by the MUR through the PRIN 2022 program under Grant 20223M4HZ4. This article was recommended by Editor-in-Chief D. Demarchi. *(Corresponding author: Luca Bompani.)*

Luca Bompani, Olmo Baldoni, and Francesco Conti are with the Department of Electrical, Electronic, Information Engineering, University of Bologna, 40126 Bologna, Italy (e-mail: luca.bompani5@unibo.it).

Luca Crupi is with the Dalle Molle Institute for Artificial Intelligence, USI-SUPSI, 6962 Lugano, Switzerland.

Daniele Palossi is with the Dalle Molle Institute for Artificial Intelligence, USI-SUPSI, 6962 Lugano, Switzerland, and also with the Integrated Systems Laboratory, ETH Zürich, 8092 Zürich, Switzerland.

Davide Brunelli is with the Department of Industrial Engineering, University of Trento, 38122 Trento, Italy.

Manuele Rusci is with the Department of Electrical Engineering, KU Leuven, 3000 Leuven, Belgium.

Luca Benini is with the Department of Electrical, Electronic, Information Engineering, University of Bologna, 40126 Bologna, Italy, and also with the Integrated Systems Laboratory, ETH Zürich, 8092 Zürich, Switzerland.

Code is available online at https://github.com/Bomps4/TAFE_Pest_Detection.

Digital Object Identifier 10.1109/TAFE.2024.3451888

I. Introduction

THE codling moth pest, scientifically known as *Cydia pomonella*, poses a significant threat to global crop production, potentially causing up to 80% fruit abscission in apple orchards [1], [2]. To prevent an excessive utilization of pesticides, it is essential to promptly detect the threat and perform localized treatments only in the affected areas [3], [4]. Given the size of the areas occupied by the orchard fields, novel automated pest detection systems have become crucial to replace costly and time-consuming human inspection [5].

Fig. 1(a) shows a trap commonly used for codling moth detection [6]. The prototype contains a sticky pad with a sex pheromone bait to attract the pests. An internal camera records the trapped insects and sends the images [e.g., Fig. 1(b)] to a remote server for analysis. The presence of the pests is assessed by detecting the codling moths using computer vision algorithms [7].

Because of the nature of the installation environment, the sensor system can only be powered by batteries or energy harvesters, e.g., solar panels. This requirement severely constrains the energy the individual electronic components, such as the image sensor, the central processing unit, and the communication module can consume. In particular, long-range transmissions of images typically require power-hungry interfaces, such as a global system for mobile (GSM) communications subsystem. As reported by Granado et al. [8] the GSM interface consumes 1.5 W in the active state, which accounts for 66% of the total system power, thus significantly impacting the lifetime of the battery-operated sensor (if we consider one image per hour it would account for ∼20 months of operation of the sensor).

Thanks to this *near-sensor processing* approach, only a few bytes of information reporting the pest counts are occasionally sent to the remote station. The low data rate enables the usage of ultra-low power protocols, such as long range (LoRa) [10], thus reducing the communication power cost compared to naive *sense-and-transmit* systems [11].

On the processing side, microcontroller units (MCUs) are commonly selected as the core devices of low-power sensor nodes [12]. These devices include single or multiple CPU cores clocked at up to a few hundred MHz and up to a few MB of on-chip memory. In our recent work [13], we showed an optimized Viola–Jones detector for pest detection on the GAP8 system-on-chip (SoC), an energy-efficient MCU featuring a compute cluster (CL) with eight general-purpose RISC-V cores.

2771-9529 © 2024 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Indian Inst of Inform Technology Guwahati. Downloaded on February 16,2026 at 06:55:30 UTC from IEEE Xplore. Restrictions apply.

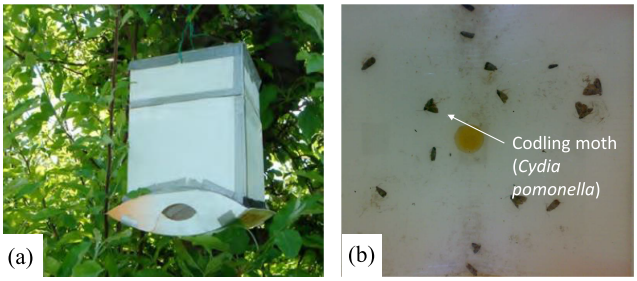


Fig. 1. (a) Trap prototype for codling moths proposed in [6] and (b) image sample taken from a trap deployed in the field [6], [9].

This method achieves a processing latency 51% lower than a previous computer vision pipeline running on the same SoC [9], which combined a background subtraction algorithm to localize the pests with a convolutional neural network (CNN) for classifying the detected objects. This article extends our precedent study [13] by analyzing traditional versus CNN-based image object detection algorithms for on-device pest detection on a state-of-the-art (SoA) MCU featuring a convolution accelerator. This class of processing devices, not yet considered in the context of pest-detection systems, has been recently introduced to speed up onboard analytics tasks by 10–100× with respect to utilizing only a single general-purpose core (e.g., ∼56× in [14] for CNN inference). *We selected the GAP9 SoC, an energy-efficient MCU that combines ten general-purpose cores with a dedicated hardware engine for CNN processing, as an example of this novel class of heterogeneous platforms*. We compare the efficient execution of our optimized Viola–Jones detector with a CNN-based image object detector, namely, MobileNetV3-SSDLite (MBNV3-SSD), featuring 3.44 million parameters and a computational load of 584 M multiply accumulate (MAC) operations. Finally, the processing unit is coupled with a low-power image sensor and a LoRa transmission module.

In summary, this article makes the following contributions:

1) we compare a traditional Viola–Jones detector versus a CNN-based approach for detecting *Cydia pomonella*;
2) we analyze latency-optimized versions of the two methods when running on the SoA GAP9 SoC;
3) we evaluate the system-level costs of the pest detection sensor using a duty-cycling power management scheme.

Our study shows that the Viola–Jones detector and MBNV3-SSD reach a similar detection rate of, respectively, 81.7% and 83% on a first pest dataset. For a second set of images with smaller-sized insects, the CNN-based detector can generalize better than Viola–Jones, achieving a detection rate of 72%, + 27% than the other method. On the GAP9 SoC, our Viola–Jones algorithm completes the detection task in 221 ms over a 320 × 240 pixel image, which is 1.47× faster than the execution time on GAP8, thanks to the higher clock speed (+37%) and the larger memory (+64 kB of low-level memory) of the most recent processing unit. On the other side, the MBNV3-SSD execution takes 147 ms when using the convolution accelerator, 9.5× faster than on the GAP8, which only uses the available general-purpose cores to run CNN workloads. This speed-up is mainly explained by the higher inference efficiency of GAP9 versus GAP8, achieving a performance level of 10.9 MAC/cycle (7.2× higher than GAP8). Thus, our study indicates that the GAP9 heterogeneous platform can enable on-board processing of accurate CNN-based pest detectors, which was previously shown only on processing units with more than 15× higher power consumption [15]. From a system-level perspective, we estimated a battery lifetime of ∼199 days if the system is powered by a 1000 mAh battery at 3.7 V and an image is analyzed every 30 s with a duty-cycled power management scheme.

II. Related Work

Computer vision for codling moth detection can be classified into two broad categories: methods running at the edge, i.e., using the processing unit inside the trap versus image analysis tasks offloaded to a remote station [5], [7]. The latter typically adopt CNN-based algorithms to analyze the images transmitted from the wireless camera sensors deployed in-field [15], [16], [17], [18], [19]. Commercial solutions, such as *TrapView*[1] and *iSCOUT*,[2] offer software packages for server-side analysis of few images per day. The system proposed by Preti et al. [11] send images to a central server through the cellular network; the pest detection relies on a particle filter detector, which based on the morphological data of *Cydia pomonella* provides regions of interests in the image, later classified by a CNN. All have been implemented with the *ImageJ* software package for image analysis [20]. Suárez et al. [21] proposed a two-stage computer vision pipeline to process the images received from the traps. A canny-edge detector is followed by a CNN for the classification task, resulting in an overall accuracy of 94.8%. In contrast to this class of methods, our solution follows a *near-sensor processing* paradigm, where image analysis is performed directly in the trap to reduce the transmission bandwidth requirements and the associated communication energy and financial cost (if a metered network is used).

Among the solutions proposed for on-board processing, *Suto* [19] combined a Raspberry PI Zero W board for image acquisition and analysis with a LoRa module to communicate the number of detected objects. The processing task consists of a selective search (SS) algorithm and a CNN classifier, MobileNetV2, leading to a total compute cycle of 195 s. Following the pipeline proposed in [18], the work by Albanese et al. [16] leveraged a CNN classifier (LeNet is the more efficient setting among the ones proposed) to classify the image patches selected by a moving window. The system revolves around a Raspberry PI 3 board, augmented with a dedicated accelerator board, the Intel neural compute stick (NCS), to speed up the CNN inference workloads. The method reached an accuracy of up to 96% at a total power cost of 5 W (3 W for the main board and 2 W for the NCS), measuring a lifetime of up to 3 months if processing two images per day and sending the count of detected objects to the server using LoRa. A similar smart trap design was proposed in [17]: a Raspberry PI 4 coupled with a 12.5 Mpixel camera runs an EfficientDet CNN-based object detector and then sends the

[1]https://trapview.com/
[2]https://metos.at/en/iscout/

Authorized licensed use limited to: Indian Inst of Inform Technology Guwahati. Downloaded on February 16,2026 at 06:55:30 UTC from IEEE Xplore. Restrictions apply.

number of detected pests. This solution achieved an accuracy of 99.3% on their test data; the system turns on once a day, reaching up to 5 W of power consumption. Despite their effectiveness, these methods rely on high-end embedded processors featuring a power consumption of several Watts, more than 10× higher than our target.

In the context of ultra-low-power systems, Schrader et al. [22] proposed to use an ESP32 device with a power envelope of 300 mW for image capturing. The final system, however, is solely employed to send the taken pictures to a central station for processing. The work in [15] used the OpenMV [23] board with a high-performance STM32H7 MCU, clocked at 480 MHz, and a VGA OV7725 image sensor. The MCU runs an SS kernel to localize the insects inside the image and a MobileNetV2 for the classification, reaching 82% accuracy. The system stays in active mode for ∼1 min a day, consuming more than 500 mW to perform image capture, object detection, and sending the pest counts through GSM.

To reduce the power consumption, the solution of Brunelli et al. [9] revolved around the ultra-low power multicore GAP8 MCU, which executes a background subtraction algorithm based on a Gaussian mixture model (GMM) to propose regions of interest that are later classified by a CNN algorithm, the latter classification algorithm achieves 93% accuracy. This system uses the long range wide area network (LoRaWAN) protocol to send the images when detection is performed, with an energy cost of up to 52 J per image. Unlike these works that rely on two-step computer vision pipelines, we recently proposed a simplified single-stage object detector based on the Viola–Jones algorithm that directly returns the pests' locations in the image [13]. On GAP8, this method takes only 400 ms to run on the 8-core CL, ∼2× faster than [9].

This article differentiates from these previous studies by comparing our traditional Viola–Jones algorithm and a CNN-based object detector, also adopted by high-end systems [15], [16], using—for the first time in the context of pest detection systems—a novel heterogeneous MCU with a specialized CNN accelerator. Thanks to the increased processing efficiency compared to the precedent MCU generation, the more complex CNN algorithm shows a similar energy consumption than the traditional detector (4.85 mJ versus 4.61 mJ), bridging the gap with more power-hungry systems.

III. Background

A. Viola–Jones

The Viola–Jones algorithm [24] scans an image with a moving window and checks for the presence of the target objects. The algorithm extracts the handcrafted Haar-like features for every image window by applying a set of rectangular filters and then runs a multistage cascade classifier. The classifier consists of many simple and computationally inexpensive tests, denoted as *weak classifiers*, which operate on the Haar-like features. Every stage of the cascade classifier includes multiple weak classifiers contributing to a final stage's score. If this score is below a threshold, the detection response on the current window is negative, and no detection is performed. On the other side, a detection is made if all the cascade stages are successfully passed. The set of filters of every stage and the threshold values are determined by a training process fed by positive and negative samples. Instead of operating on the image space, the original paper proposed an intermediate representation, denoted as the *Integral Image* II, to compute the Haar-like features efficiently. This integral image stores at every location (x, y) the sum of the pixels of the input image, i.e., the total brightness level, in the rectangular area delimited by (0,0) and (x, y). For example, in Fig. 2, the pixel A in the integral Image, $II[A]$, is the sum of the pixels in the area A of the input image. By precomputing the integral image, we can easily calculate the integral of any arbitrary area in the input image, which is the founding operation for the Haar-like features.

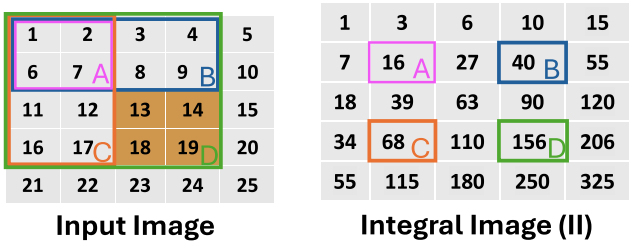


Fig. 2. Example of the input image (left) and the corresponding integral image (on the right). The values (A)–(D) of the integral image correspond to the integral of the areas (A)–(D) in the input image (rectangles with matching letters and colors). By precomputing the integral image, the integral value of the brown area can be computed as II[A] -II[B]-II[C] +II[D].

For example, the integral of the brown area in Fig. 2, is simply computed as $II[D] + II[A] - II[B] - II[C] = 156 + 16 - 40 - 68 = 32$: the operation requires only four read accesses in the memory independently of the size of the integral area.

B. MobileNetV3-SSDLite

Following the analysis performed in [25], we selected the MobilenetV3-SSDLite model for our pest detection task as it represents the best compromise between accuracy and network complexity. Other models with higher accuracy scores cannot fit the memory resources (e.g., Retina-Net [26]) of our platforms or do not feature predictable execution times (e.g., Faster R-CNN [27]), which is essential for real-time embedded devices. MobileNetV3-SSDLite is a CNN-based object detector that is composed of a feature extractor, also denoted as the *backbone*, and a regression (or classification) head [28]. More in detail, this model uses MobileNetV3 [29] as the feature extractor and the SSDLite as the regression head [28]. Both the subnetworks consist of multiple convolution layers. Each layer applies a convolution filter over an input feature map, typically denoted as the *activation* tensor. The activation tensor output is then passed to the next layer. Hence, the final network result is obtained by propagating data through all the layers, also called the *inference* task.

The MobileNetV3 and the SSDLite blocks rely on depthwise-separable convolution as the fundamental operator. Concerning standard 2-D convolutional filters, a depthwise-separable convolution is composed of a depthwise and pointwise convolution, reducing the number of filter parameters and operations by 86.8% in the case of a 3×3 convolution with 16 input and

Authorized licensed use limited to: Indian Inst of Inform Technology Guwahati. Downloaded on February 16,2026 at 06:55:30 UTC from IEEE Xplore. Restrictions apply.

output channels. More in detail, the MobileNetV3 architecture includes inverse residual blocks, which expands the number of input features with a pointwise convolution. The produced feature maps are then passed to a depthwise convolution and finally compressed back with another pointwise layer. A residual connection is added when the input feature size matches the size of the output tensor. On the other side, SSDLite comprises four depthwise-separable convolution blocks, each attending a diverse object size. This regression head takes the features produced by the backbone and produces the bounding boxes and classes of the object inside the image.

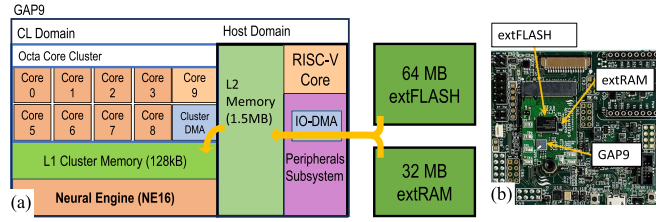


Fig. 3. (a) Block diagram of the GAP9 SoC and the interfaces with the external memories. The yellow arrows indicate the data copies between the L1 and L2 memories and between the off-chip memories and the L2 memory operated by, respectively, the CL DMA and the IO-DMA. (b) GAP9 carrier board.

IV. Methodology

A. On-Device Processing Requirements

To determine the throughput requirement for on-device pest detection, we examine the operational role of a smart trap deployed in the field. Smart traps are instrumental in relaying data on pest activity, which is crucial for implementing timely and effective pest control measures. Based on this, we define the following two application scenarios dictating the latency requirement for the processing task.

1) *Low-energy scenario:* In this scenario, we perform detections at the same frequency as pheromone sprays, one every 15 min [30], ensuring our platform's responsiveness while minimizing energy consumption. Thanks to this, we can provide information on pest activity, which can be used to optimize pheromone spraying, releasing higher concentrations only when pest activity is detected and reducing unnecessary pheromone waste, as proposed in [31].
2) *High-frequency scenario:* In this scenario, we increase the detection frequency to one every 30 s to address the limitations of current pest management methods, which rely on daily assessments of pest presence and temperature [32], [33]. This higher detection rate fills a data gap, allowing for more accurate analysis by correlating pest activity with environmental factors, such as airspeed and humidity. Since the presence of even a single moth can trigger pest disruption methods [34], the capability to detect each new moth entering the trap is necessary to improve our understanding of pest behavior and improve current pest management techniques.

In both scenarios, the system must immediately transmit the pest count after detection to activate the pheromone dispensers. From a system-level viewpoint, we target a battery-powered node that can operate for the whole activity period of the Cydia pomonella, around 4 months [33]. Due to its cost-effectiveness and low cost (less than 10 dollars), we have selected a 1000 mAh battery as a system requirement because this type of battery is commonly used in low-power sensor nodes [35], [36] and internet of things (IoT) sensor nodes [37].

B. Heterogeneous Processing Platform

We use the GAP9 SoC by GreenWaves Technologies as the main processing unit for on-board pest detection. The microarchitecture of GAP9, which is schematized in Fig. 3(a), is composed of a *host* domain and a programmable accelerator subsystem, denoted as CL. The *host* domain includes a RISC-V programmable core, a 1.5 MB L2 memory, and a broad set of peripherals. Conversely, the CL combines 9 RISC-V cores with a dedicated convolution accelerator, called NE16, to accelerate the execution of compute-intensive linear algebra kernels, e.g., CNN's convolution operations.

The RISC-V cores feature vectorial MAC instructions that execute a dot-product between two 4×8-bit vectors and accumulate the results in a single clock cycle. The NE16 comprises instead $9 \times 9 \times 16$ 8×1 bit MAC units, reaching a peak compute efficiency of 150 8-bit MAC operations per clock cycle, up to $\sim 5\times$ faster than using the CL cores. The CL domain also features a low-level L1 scratch memory of 128 kB that can be accessed in a single-clock cycle from the cores and NE16. These engines can also access data stored in the larger L2 memory but with a $\sim 10\times$ lower bandwidth than accessing the L1 memory. Alternatively, a CL DMA engine can be programmed to copy data between the L1 and the L2 memories in the background of the core operations.

At the board level [see Fig. 3(b)], the chip is interfaced with an external 32 MB RAM and a 64 MB FLASH memory via octaSPI. An IO-DMA within the peripheral subsystem of the GAP9's *host* domain is responsible for copying data between the external and on-chip L2 memory with a maximum data rate of 1 byte per clock cycle. 1-D or 2-D memory transfers can be programmed via software; the software call of a 2-D data copy enqueues multiple 1-D copies. In GAP9, the *host* and the CL reside on separate voltage and frequency domains, enabling fine-grained dynamic voltage and frequency scaling. To gain maximum energy efficiency, we configure a voltage level of 0.65 V and a clock frequency of 240 MHz for both domains.

Table I summarizes the main characteristics of the GAP9 and highlights the main differences concerning the precedent MCU generation, the GAP8 SoC, which was used for image-based pest detection by some recent works [9], [13]. Notably, GAP9 features a higher on-chip memory budget and a specialized hardware engine to speed up CNN inference workloads absent on GAP8.

C. Accelerating Pest Detection on GAP9

This section describes the optimized deployment strategies of the pest detection algorithms on GAP9. Both the Viola–Jones and the MBNV3-SSD CNN detectors are trained on a custom

Authorized licensed use limited to: Indian Inst of Inform Technology Guwahati. Downloaded on February 16,2026 at 06:55:30 UTC from IEEE Xplore. Restrictions apply.

TABLE I
COMPARISON BETWEEN THE GAP8 AND THE GAP9 ARCHITECTURES

	GAP8	GAP9
RISC-V Cores *Host* + *Cluster*	1+8	1+9
On-chip L1 memory	64 kB	128 kB
On-chip L2 memory	512 kB	1.5 MB
CNN Accelerator	No	NE16
Floating point hardware support	No	Yes
VDD	1.2 V	0.65 V
Max Clock Frequency *Host*	250 MHz	240 MHz
Max Clock Frequency *Cluster*	175 MHz	240 MHz
Fabrication Technology	65 nm	22 nm
Peak Efficiency	4.24 mW/GOP	0.33 mW/GOP

dataset composed of 158 images (320 × 240 pixel) with a total of 1275 target insects manually labeled. All the images were taken using an RGB sensor with a resolution of 2048 × 1536 pixels under daylight conditions (from 8 am to 7 pm) without employing any external illumination. Because the collected images contain 27 nontarget objects (<1.6 % of the total amount of pests), of which only 3 in the test split, our detection algorithms are designed to identify only objects belonging to the *Cydia pomonella* category.

1) Viola–Jones: A 15-stage cascade classifier is trained using a variant of the AdaBoost algorithm [24]. The training set is composed of the 20 × 20 patches of *Cydia pomonella* cropped from the available images and a set of negative crops from a different image dataset available online.[3] To account for the diverse sizes of the target objects, we run the Viola–Jones algorithm over five scales of the original images. At every iteration, the image resolution is scaled down by a factor of 1.1 in horizontal and vertical dimensions. Detections larger than 30 × 30 pixels are discarded.

To run the Viola–Jones detector on the GAP9 MCU efficiently, we use the parallel software code presented in our recent work [13]. In this implementation, the input image is stored in the L2 memory, occupying 76 kB (320 × 240 `INT8`). Unfortunately, the L1 memory is not large enough to fit the input data and the integral image, which demands a total of 307 kB (320 × 240 × 4 byte/elements – `INT32` values). To solve this problem, we split the original image (and the scaled versions) into tiles of size 100 × 240 pixels. Every tile is copied from the L2 to the L1 memory at runtime and then processed. To account for potential detections between adjacent tiles, we set an overlap of 20 pixels in the tiling logic for both the horizontal and vertical directions.

The processing kernel takes an input data tile to compute a tiled version of the integral image, which features a size of 96 kB and is stored in the L1 memory. The CL cores then scan (in parallel) different regions of the image tile with a 20 × 20 sliding window. Every core invokes the cascade classifier to test if the current window includes the target object. When a stage of the classifier returns a negative response, the evaluation continues to the next window of the tile. The parallelization is operated over eight CL cores; the ninth core works as the task dispatcher. At the end of the analysis, the algorithm outputs a list of detections described by their bounding box coordinates.

2) Convolutional Neural Network: The MobileNetV3-SSDLite object detection network features 3.44 M parameters and accounts for 584 M MAC operations to process a 320 × 240 image. Similarly to Viola–Jones, this detector outputs a set of bounding boxes, each associated with a predicted class, i.e., *Cydia Pomonella* versus *background*. The model is trained for five epochs using our custom dataset with an 80%–20% split between training and validation, resulting in 127 and 31 images, respectively. stochastic gradient descent is the optimizer with a momentum of 0.9, a weight decay of 5×10^{-4}, and a learning rate of 0.005. The weights of the MobileNetV3 backbone are initialized from a model pretrained on the COCO dataset [38]; the parameters of the SSDLite heads are instead learned from scratch. To deploy the CNN detector on GAP9, we use the *GAP*flow toolset.[4] This software package takes the CNN model description (`ONNX` format) and generates the inference `C` code. To use the NE16 accelerator, the model is first quantized to 8-bit using the post-training quantization routine of the *GAP*flow, which takes a few samples from the training set (2 in our case) to estimate the quantization range, i.e., the *calibration* process. The 8-bit quantized parameters are then stored in the external FLASH memory.

At runtime, the inference program follows a layer-by-layer processing schedule. For this task, we allocate one memory buffer in the L2 memory to store the layer operands and a buffer in the L1 memory, serving to store temporary results. The sizes of these buffers are, respectively, 1.2 MB and 115.6 kB. Depending on the layer requirement, the program can transfer the weight parameters from the external FLASH memory to the on-chip L2 memory before the layer execution. Conversely, a layer's input and output activation tensors may be allocated in the on-chip L2 fast memory or the external RAM.

Every layer function copies data (weights and activations) from the L2 (or external) memory to the L1 buffer. It dispatches the processing job, e.g., a convolution operation, on the NE16 engine or the general-purpose cores. The latter option is selected if no convolution accelerator is available, like in the GAP8 SoC, or if the accelerator does not support that layer. We also highlight the impact of the L1 and L2 buffer sizes on the inference throughput. Generally, a larger L2 memory buffer favors allocating more tensors on the on-chip memory, leading to a faster read and write time than accessing data to/from off-chip memories. At the same time, an increased L1 buffer reduces the number of copies from the L2 memory, lowering the associated latency overheads.

D. System-Level Power Management

The final pest detection system includes the GAP9 processing unit, a Himax HM01B0 ultra-low-power camera sensor, which captures 320 × 240 pixels 8-bit grayscale images, and a LoRa module, i.e., the HTCC-AB01 board by CubeCell, connected via UART with the MCU. The latter relies on the AM01 module, which incorporates a low-power cortex M0 MCU with

[3]https://pythonprogramming.net/static/images/opencv/negative-background-images.zip

[4]https://greenwaves-technologies.com/tools-and-software/

Authorized licensed use limited to: Indian Inst of Inform Technology Guwahati. Downloaded on February 16,2026 at 06:55:30 UTC from IEEE Xplore. Restrictions apply.

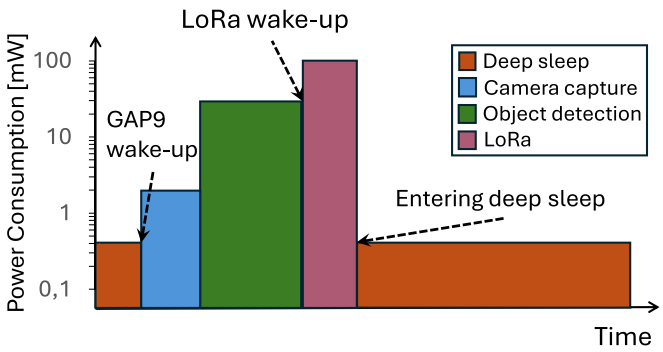


Fig. 4. System-level power management strategy. The system periodically wakes up from deep sleep mode to capture and process an image. Then, the output is sent via LoRa before going back to deep sleep.

TABLE II
DETECTION ACCURACY ON THE *NEAR-DS* AND *FAR-DS* DATASETS

Method	Quantization	*near-ds*	*far-ds*
Viola-Jones	`int8`	81.7 %	45.0 %
MBNV3-SSD	`float`	88.0 %	84.0 %
MBNV3-SSD	`int8`	83.0 %	72.0 %

16 kB of RAM running at 48 MHz and the SX1262 transceiver implementing the physical frequency modulation of the LoRa standard.

Fig. 4 shows the used power management scheme to minimize the system energy consumption. Following a duty-cycling scheme, the MCU wakes from deep sleep using the internal real-time clock (RTC). In the active state, the system captures an image and runs the pest detection algorithm. Lastly, the GAP9 MCU activates the HTCC-AB01 board module to transmit the count of detected insects using the LoRa protocol at an energy cost of 1 mJ per byte sent. Once the transmission is completed, the GAP9 receives an acknowledgment signal from the HTCC-AM01 module and returns to the deep sleep mode. We use this same execution scheme for both our low-energy and high-frequency scenarios.

V. EXPERIMENTAL RESULTS

A. Detection Accuracy

We tested the effectiveness of the Viola–Jones and the MBNV3-SSD detectors on two test sets, namely *near-ds*, which includes 11 samples with 196 target objects, and *far-ds*, composed of seven images for a total of 158 objects. The two datasets differ by the distance between the sticky pad and the camera sensor. All the images were taken using an RGB sensor with a resolution of 2048 × 1536 pixels. For a fair evaluation of the characteristics of the low-power sensor of our system, we resize the images of both datasets to 320 × 240 pixels and apply a grayscale transformation with additive Gaussian noise to mimic the image quality of the Himax camera sensor.

Table II reports the detection accuracies scored by Viola–Jones and MBNV3-SSD when in full-precision (`float`) or quantized to 8-bit (`int8`). Our metric assesses a correct detection if the predicted bounding box overlaps a manual annotation (the IoU score used by the COCO metrics [39] is set to 0.01). The Viola-Jones algorithm achieves a detection accuracy of 81.7% on the *near-ds* but only scores 45% on the *far-ds* dataset. We motivate this gap with the poor generalization capacity of the Viola–Jones algorithm: the training set shows a higher correlation with the *near-ds* than the *far-ds* set. On the other side, the `float` baseline of the MBNV3-SSD detector reaches a superior detection accuracy compared to Viola–Jones in both datasets: 88.0% and 84.0% for, respectively, the *near-ds* and *far-ds*. The post-training quantization process introduces an accuracy drop, reaching −12.0% for the *far-ds* case. On the other dataset, the degradation in performance is limited to −5%, possibly also explained by the reduced difference between the training and test domains. As an illustrative example, Fig. 5 shows the outputs of the `float` vs. `int8` models on reference samples from the two sets, highlighting the miss-detections of the quantized model.

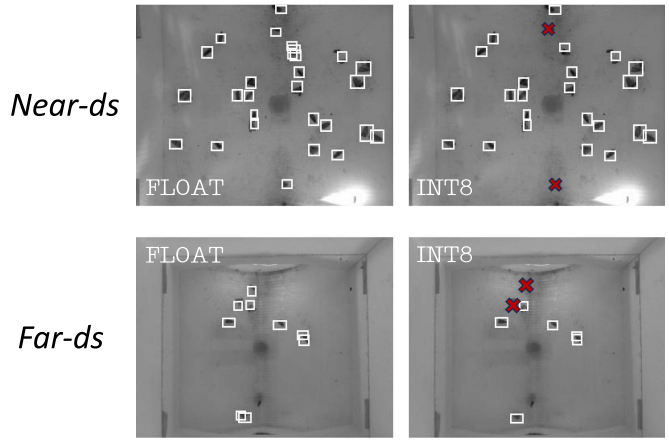


Fig. 5. CNN-based pest detection on the image samples from the *near-ds* (upper row) and *far-ds* (bottom row) testsets. Outputs from the baseline `float` model (left) are visually compared with the detections of the `int8` quantized model. The red crosses mark the miss-detections.

TABLE III
MEMORY FOOTPRINT AND LATENCY

Method	Platform	L1 [kB]	L2 [kB]	extRAM [MB]	extFLASH [MB]	Latency [M cycles]	MAC/ cycle
Viola-Jones	GAP8 (CL)	51.6	384	–	–	55.0	–
	GAP9 (CL)	99.6	384	–	–	52.3	–
MBNV3-SSD	GAP8 (CL)	47.6	267	1.6	3.44	255.3	1.5
	GAP9 (NE16)	115.6	1200	1.3	3.44	35.3	10.9

B. Latency Analysis

Table III reports the memory usage and the latency measured when executing the Viola–Jones and MBNV3-SSD on the GAP9 SoC with the NE16 accelerator. For comparison purposes, we also report the measurements on the GAP8 MCU, where the CNN inference task runs on the eight general-purpose cores of the CL (indicated by CL in the table). Both chips are configured in their best energy-efficient point: the clock frequency of GAP9 is set to 240 MHz with an operating voltage of 0.65 V while GAP8 operates at 175 MHz at 1.2 V.

Authorized licensed use limited to: Indian Inst of Inform Technology Guwahati. Downloaded on February 16,2026 at 06:55:30 UTC from IEEE Xplore. Restrictions apply.

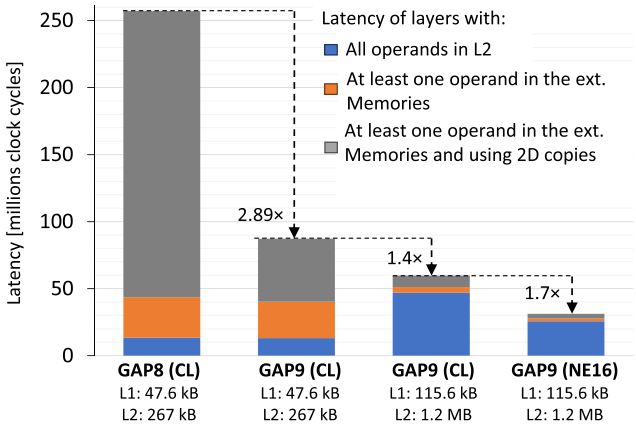


Fig. 6. Latency analisys of the MobileNetV3-SSDLite execution on GAP8 and GAP9 under varying L1 and L2 memory budgets.

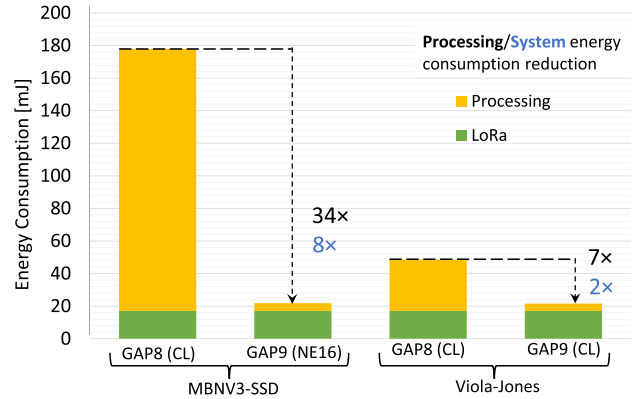


Fig. 7. Energy consumption of the system during an active cycle composed of camera acquisition, processing, and transmission of the result.

The Viola-Jones algorithm requires 384 kB of L2 memory for the input and the integral image. The L1 memory stores an image tile (the tiles size is 100 × 240 on GAP9 versus 100 × 120 on GAP8), with a maximum occupation of 99.6 and 51.6 kB, for GAP9 and GAP8, respectively. The parameters of the cascade classifier, also preserved in the low-level memory, occupy only 3.56 kB. No external memory is required. On the other hand, MBNV3-SSD uses the off-chip FLASH memory to permanently store the network parameters for a total of 3.44 MB. As described in Section III-B, we allocate an L2 buffer for (part of) the parameters, the activation tensors, and an L1 array acting as the working memory. In addition, an array of 1.6 MB is allocated in the external RAM for the input and output activation tensors not fitting into the L2 memory.

Table III also reports the number of clock cycles spent for the algorithm executions. For the CNN, we also detail the ratio between MAC and clock cycle (MAC/cycle in the table) as an efficiency indicator. Overall, the execution is 1.43× and 9.5× faster on GAP9 than executing, respectively, Viola–Jones and MBNV3-SSD on GAP8. On the latter device, our parallel Viola–Jones software outperforms the CNN execution by 4.43×. Thanks to the CNN accelerator, GAP9 achieves a processing time of 147 ms for the MBNV3-SSD execution, 1.5× lower than the Viola–Jones latency. Fig. 6 gives insights into the CNN inference latency measurements on GAP8 and GAP9 when varying the sizes of the L2 and L1 buffers and when using the general-purpose cores or the NE16 accelerator as the principal compute engine (indicated, respectively, with CL and NE16in the figure). In the plot, we break down the processing time with respect to the memory locations of the operands of the different layers. More in detail, we distinguish between the total latency of the layers whose inputs and outputs are in the L2 memory (indicated as L2 on the plot, in blue) and the layers that fetch input data from the external memory, highlighting the cases that use 1-D or 2-D memory transfers, respectively, in orange and gray.

When comparing the GAP8 versus GAP9 execution in the CL setting under the same memory budget (46.7 kB of L1 and 267 kB of L2), the higher clock cycle count on GAP8 is due to not-optimized 2-D memory transfers from the external memory (4.6× slower read time). When increasing the memory sizes of the L2 and L1 buffers on GAP9, a 1.4× speed-up is achieved mainly thanks to the higher bandwidth of the on-chip versus the external memory. In this case, the workload of the layers whose inputs are in the L2 memory dominates the total number of operations (93% of the total) and the total execution time (79% of the clock cycles). These layers' MAC/cycle ratio also increases by +10% on average concerning the low-memory configuration due to the larger tensors stored in the L1 memory buffer. Lastly, the NE16 accelerator further boosts the system efficiency by 1.7×. The speed-up is lower than the theoretical peak because of two main reasons. The nonlinear *Hsigmoid* function featured by MobileNetV3 is not natively supported by the accelerator. Hence, it must be executed on the general-purpose cores. Second, the depthwise convolutions underuse by 1/16 of the NE16 compute engine.

C. Energy Analysis

In Fig. 7, we estimate the energy consumption of our system during an active cycle that includes the camera capture, the object detection, and the LoRa wireless transmission. In the plot, we break down the energy costs for the image sensor, the MCU, namely, GAP8 or GAP9 running Viola–Jones or the MBNV3-SSD, and the LoRA module described in Section IV-D. First, the energy consumption of the camera sensor is negligible with respect to the other system costs. On GAP8, the CNN compute cost is ∼10× higher than the transmission cost (161 mJ versus 17 mJ). We account for the radio's cost to transmit 4 bytes for the pest count and 13 bytes of the LoRaWAN protocol overhead. With Viola–Jones, the energy consumption gap decreases, but the processing (31.8 mJ) still presents the largest energy cost. On the other hand, the energy consumption of the GAP9 SoC, which we measured on an evaluation board comprising the external memories, is 7× and 34× lower than the energy consumption of the GAP8 MCU for the execution of, respectively, Viola–Jones and MBNV3-SSD. The 59% of this improvement is explained by the micro-architectural innovations of GAP9 versus GAP8, i.e., more memory and the NE16 accelerator, reducing the clock cycle count.

Authorized licensed use limited to: Indian Inst of Inform Technology Guwahati. Downloaded on February 16,2026 at 06:55:30 UTC from IEEE Xplore. Restrictions apply.

TABLE IV
COMPARISON OF PEST DETECTION CAMERA-BASED SYSTEMS WITH ULTRA-LOW POWER MCUS FOR ONBOARD PROCESSING

	Method	Processor	Pest Detections Latency	Processing Cost [mJ]	Comm. Module	Daily energy consumtpion [J]			
						High-frequency		Low-energy	
						Transmitting images	Transmitting pest counters	Transmitting images	Transmitting pest counters
J.Suto [15]	DNN (Parameters: 3.2 M) Acc:82%	STM32H7 480 MHz	1 min	30600	GSM images	—	—	2937.6	—
Brunelli et al. [9]	GMM+DNN (Parameters:ND) Acc:93%[a]	GAP8 50 MHz	810 ms + 51 ms/inference	**3.5** [b]	LoRaWAN images	1728.7	—	1719.0	—
Rusci et al. [13]	Viola-Jones (Parameters: 3.6 k) Acc:81.7%	GAP8 175 MHz	400 ms	31.8	LoRaWAN pest counter / images	513.2	143.1	424.8	7.7
This Work	Viola-Jones (Parameters: 3.6 k) Acc:81.7%	GAP9 240 MHz	221 ms	4.61		**437.5**	**66.9**	**423.3**	**5.8**
	MBNV3-SSD (Parameters:3.44 M) Acc:83%	GAP9 (NE16) 240 MHz	**147 ms**	4.85		445.0	74.4	423.4	5.9

[a] only tested on crops, [b] not accounting for external memories,

The rest can be instead attributed to the lower power consumption of GAP9, reaching 20.5 mW and 33 mW for Viola–Jones and the CNN processing, the latter including also the energy consumption of the external memories. On GAP8, we measured a power consumption of 79 mW instead. From a system-level viewpoint, the energy cost of the GAP9 processing unit is approximately $\sim 3.5\times$ lower than the transmission cost, thus enabling energy-efficient near-sensor pest detection with a minimal impact on the overall energy cost. This solution presents a consumption higher of only 0.24 mJ with respect to the lightweight Viola–Jones algorithm while providing a higher accuracy (+1.3% and +27%, respectively, for the *near-ds* and *far-ds*)

D. Comparison versus SoA

We evaluate the energy consumption of our method in a complete system composed of a camera, the MCU with its external memories, and a LoRa transmitter as described in Section IV-D. Table IV compares our solution with other SoA camera-based systems for codling moth detection that use MCUs for onboard processing. The table outlines the computer vision algorithms employed and their processing and transmission costs during the active state. It also details the total daily energy consumption, which accounts for the energy spent in active and deep sleep states, and the energy needed to load the DNN's weights from the external to the on-device memory. We adopt a transmission policy for the system-level evaluation that sends the pest count after detection, acting as a live report within the final application scenario.

The method by *Suto* [15] reaches a detection accuracy of 82%, similar to our solution, despite not being directly comparable because of the diverse dataset (not publicly available). Their MCU, the STM32H7, consumes 30 J of energy for a MobileNetV2 inference, 6300× higher than our best solution executing a similar workload. Their MCU, the STM32H7, consumes 30 J of energy for a MobileNetV2 inference, 6300× higher than our best solution executing a similar workload, leading to a daily consumption 6.9× higher than our CNN and a battery lifetime of only 4.6 days in the low-energy scenario. Furthermore, the platform requires 1 min to perform the inference, which does not match the requirements of the high-frequency scenario.

In contrast, the accuracy score (93%) reported in [9] only refers to the classification of a set of patches extracted from the frames of our same dataset. However, the background subtraction algorithm, which serves as a patch extractor, can lead to the misidentification of significant patches, thereby compromising the overall accuracy of the pest count. Variations in luminosity can result in the misclassification of these patches, a problem to which CNNs have demonstrated greater robustness [40]. Unlike background subtraction, which relies solely on detecting differences between frames, CNNs do not suffer from the inability to recover from false negatives. If a pest is not detected in the initial frame when it enters the trap, the background subtraction algorithm may subsequently classify it as part of the background, preventing its detection in future frames. In contrast, CNNs, which analyze each frame independently, are not subject to this limitation. For these reasons, the detection accuracy of our method is not directly comparable with the score reported in [9]. A fairer comparison between our methods is to apply their CNN, patch by patch, to the whole image, giving us an energy consumption of 31.5 mJ (6.6× more than ours) Lastly, our object detection algorithm deployed on the GAP9 platform consumes 4.85 mJ of energy. This is only 1.35 mJ more than the energy consumption reported in [9], which used a low-frequency operating point (50 MHz) on the GAP8 platform. However, it is important to note that [9] did not account for the energy cost of the external memories.

Since the system described in [9] transmits image patches upon detection, its daily energy consumption is 1728.7 and 1719.0 J for the high-frequency and low-energy scenarios, respectively. In this estimate, we considered the transmission cost from [9] and an average of approximately 33 codling moths detected per trap per day, as reported in [41]. To fairly compare this system with our work, we calculated the energy consumption in the case our node sends an image after every detection (a total payload of 12.7 kB that includes a JPEG-compressed image and the overhead of the LoRaWAN protocol). Given that our LoRa module consumes 1 mJ per byte sent, the total energy consumption is 12.7 J per image. For 33 detections per

Authorized licensed use limited to: Indian Inst of Inform Technology Guwahati. Downloaded on February 16,2026 at 06:55:30 UTC from IEEE Xplore. Restrictions apply.

day, as reported in [41], our system's daily energy consumption is 445.0 J in the high-frequency scenario and 423.4 J in the low-energy scenario. This daily energy consumption is approximately 3.9× lower than the system described in [9] under the same conditions. Considering that we send only pest counters with the same frequency of detections, the energy consumption of our system is reduced by 6.5 times compared to sending the frame if a detection is made, achieving a lifetime of 199 days.

GAP9 achieved a 7× reduction in processing energy compared to our previous work [13], which used GAP8 as the main processing unit. This significant reduction in processing cost decreases the daily energy consumption by 76.2 J for high-frequency and 1.9 J for low-energy scenarios. The proposed system can achieve a lifetime of 170 days for MBNV3-SSD and 199 days for Viola–Jones in the high-frequency scenario, and 2257 days for MBNV3-SSD and 2296 days for Viola-Jones in the low-energy scenario when sending pest counts at the same frequency as detections. In both scenarios, this exceeds the expected activity period of Cydia pomonella of approximately 120 days [33].

VI. Conclusion

In this work, we deployed a lightweight Viola–Jones and a CNN-based MobileNetV3-SSDLite algorithm for pest detection on a SoA multicore MCU, the GreenWaves Technologies' GAP9, featuring a convolution hardware accelerator. We compare the performances of both methods in terms of efficiency and accuracy. On two datasets, namely, *near-ds* and *far-ds*, Viola-Jones achieved a detection accuracy of, respectively, 81.7% and 45%. The accuracy of this lightweight detector is outperformed by MBNV3-SSD in both datasets, scoring 83% and 72%, respectively.

Conflict of Interest

The authors of the manuscript declare no conflict of interest. Brand names and/or any mention or listing of specific commercial products or services herein are used solely for reference or comparison purposes. Such use does not imply endorsement by any of the manuscript's authors or affiliations.

Acknowledgment

The authors would like to thank Marco Fariselli, Tommaso Polonelli, and Carlo Montanari for their support.

References

[1] D. Ju, D. Mota-Sanchez, E. Fuentes-Contreras, Y.-L. Zhang, X.-Q. Wang, and X.-Q. Yang, "Insecticide resistance in the cydia pomonella (l): Global status, mechanisms, and research directions," *Pesticide Biochem. Physiol.*, vol. 178, 2021, Art. no. 104925.

[2] F. Wan et al., "A chromosome-level genome assembly of cydia pomonella provides insights into chemical ecology and insecticide resistance," *Nature Commun.*, vol. 10, no. 1, 2019, Art. no. 4237.

[3] E. Roma, V. A. Laudicina, M. Vallone, and P. Catania, "Application of precision agriculture for the sustainable management of fertilization in olive groves," *Agronomy*, vol. 13, no. 2, 2023, Art. no. 324.

[4] G. Kamyshova et al., "Artificial neural networks and computer vision's-based phytoindication systems for variable rate irrigation improving," *IEEE Access*, vol. 10, pp. 8577–8589, 2022.

[5] J. Suto, "Codling moth monitoring with camera-equipped automated traps: A review," *Agriculture*, vol. 12, no. 10, 2022, Art. no. 1721.

[6] A. Guarnieri et al., "Automatic trap for moth detection in integrated pest management," *Bull. Insectol.*, vol. 64, no. 2, pp. 247–251, 2011.

[7] A. C. Teixeira, J. Ribeiro, R. Morais, J. J. Sousa, and A. Cunha, "A systematic review on automatic insect detection using deep learning," *Agriculture*, vol. 13, no. 3, 2023, Art. no. 713.

[8] O. López Granado, M. Martínez-Rach, H. Migallón, M. Malumbres, A. Bonastre Pina, and J. Serrano Martín, "Monitoring pest insect traps by means of low-power image sensor technologies," *Sensors*, vol. 12, pp. 15801–158019, Dec. 2012.

[9] D. Brunelli, T. Polonelli, and L. Benini, "Ultra-low energy pest detection for smart agriculture," in *Proc. IEEE SENSORS*, 2020, pp. 1–4.

[10] M. Saari, A. M. Bin Baharudin, P. Sillberg, S. Hyrynsalmi, and W. Yan, "LoRa – A survey of recent research trends," in *41st Int. Conv. Inf. Commun. Technol., Electron. Microelectron.*, 2018, pp. 0872–0877.

[11] M. Preti, C. Moretti, G. Scarton, G. Giannotta, and S. Angeli, "Developing a smart trap prototype equipped with camera for tortricid pests remote monitoring," *Bull. Insectol.*, vol. 74, no. 1, pp. 147–160, 2021.

[12] D. Singh, S. Sai Prashanth, S. Kundu, and A. Pal, "Low-power microcontroller for wireless sensor networks," in *Proc. IEEE Region 10 Conf.*, 2009, pp. 1–6.

[13] M. Rusci, L. Bompani, O. Baldoni, C. Montanari, D. Brunelli, and L. Benini, "Parallel execution of the Viola-Jones algorithm on MCUS for low-cost automated pest detection," in *2023 IEEE Conf. AgriFood Electron.*, 2023, pp. 35–39.

[14] F. Conti et al., "Marsellus: A heterogeneous RISC-V AI-IoT end-node SoC with 2–8 b DNN acceleration and 30%-boost adaptive body biasing," *IEEE J. Solid-State Circuits*, vol. 59, no. 1, pp. 128–142, Jan. 2024.

[15] J. Suto, "Embedded system-based sticky paper trap with deep learning-based insect-counting algorithm," *Electronics*, vol. 10, no. 15, 2021, Art. no. 1754.

[16] A. Albanese, M. Nardello, and D. Brunelli, "Automated pest detection with DNN on the edge for precision agriculture," *IEEE Trans. Emerg. Sel. Topics Circuits Syst.*, vol. 11, no. 3, pp. 458–467, Sep. 2021.

[17] D. Čirjak, I. Aleksi, D. Lemic, and I. Pajač Živković, "EfficientDET-4 deep neural network-based remote monitoring of codling moth population for early damage detection in apple orchard," *Agriculture*, vol. 13, no. 5, 2023, Art. no. 961.

[18] W. Ding and G. Taylor, "Automatic moth detection from trap images for pest management," *Comput. Electron. Agriculture*, vol. 123, pp. 17–28, 2016.

[19] J. Suto, "A novel plug-in board for remote insect monitoring," *Agriculture*, vol. 12, no. 11, 2022, Art. no. 1897.

[20] C. A. Schneider, W. S. Rasband, and K. W. Eliceiri, "NIH image to Imagej: 25 years of image analysis," *Nature Methods*, vol. 9, no. 7, pp. 671–675, Jul. 2012.

[21] A. Suárez, R. S. Molina, G. Ramponi, R. Petrino, L. Bollati, and D. Sequeiros, "Pest detection and classification to reduce pesticide use in fruit crops based on deep neural networks and image processing," in *2021 XIX Workshop Inf. Process. Control*, 2021, pp. 1–6.

[22] M. J. Schrader, P. Smytheman, E. H. Beers, and L. R. Khot, "An open-source low-cost imaging system plug-in for pheromone traps aiding remote insect pest population monitoring in fruit crops," *Machines*, vol. 10, no. 1, 2022, Art. no. 52.

[23] I. Abdelkader, Y. El-Sonbaty, and M. El-Habrouk, "Openmv: A python powered, extensible machine vision camera," 2017, *arXiv:1711.10464*.

[24] P. Viola and M. Jones, "Rapid object detection using a boosted cascade of simple features," in *Proc. IEEE Comput. Soc. Conf. Comput. Vis. Pattern Recognit.*, 2001, vol. 1, pp. I–I.

[25] B. Luca, F. Alberto, J. Mauro, P. Mauro, and A. Cesare, "Precise agriculture: Effective deep learning strategies to detect pest insects," *IEEE/CAA J. Automatica Sinica*, vol. 9, no. 2, pp. 246–258, Feb. 2022.

[26] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, "Focal loss for dense object detection," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 42, no. 2, pp. 318–327, Feb. 2020.

[27] S. Ren, K. He, R. Girshick, and J. Sun, "Faster R-CNN: Towards real-time object detection with region proposal networks," in *Proc. Int. Conf. Adv. Neural Inf. Process. Syst.*, 2015, pp. 91–99.

[28] W. Liu et al., "SSD: Single shot multibox detector," in *Proc. Eur. Conf. Comput. Vis.*, B. Leibe, J. Matas, N. Sebe, and M. Welling, Eds. Cham, Switzerland: Springer, 2016, pp. 21–37.

[29] A. Howard et al., "Searching for mobilenetv3," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, 2019, pp. 1314–1324.

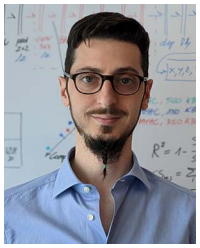
Authorized licensed use limited to: Indian Inst of Inform Technology Guwahati. Downloaded on February 16,2026 at 06:55:30 UTC from IEEE Xplore. Restrictions apply.

[30] M. Baldessari, C. Rizzi, G. Tolotti, and G. Angeli, "Evaluation of an aerosol emitter for mating disruption of cydia pomonella in Italy," *Commun. Agricultural Appl. Biol. Sci.*, vol. 78, pp. 267–71, Jan. 2013.
[31] B. Giovanni, L. Andrea, D. Thomson, and C. Ioriatti, "Sex pheromone aerosol devices for mating disruption: Challenges for a brighter future," *Insects*, vol. 10, Sep. 2019, Art. no. 308.
[32] P. L. Shaffer and H. J. Gold, "A simulation model of population dynamics of the codling moth, cydia pomonella," *Ecological Modelling*, vol. 30, pp. 247–274, 1985.
[33] P. Damos and N. Kouloussis, "A degree-day phenological model for cydia pomonella and its validation in a mediterranean climate," *Bull. Insectol.*, vol. 71, pp. 131–142, 2018.
[34] V. P. Jones et al., "Predicting the emergence of the codling moth, cydia pomonella (lepidoptera: Tortricidae), on a degree-day scale in North America," *Pest Manage. Sci.*, vol. 69, no. 12, pp. 1393–1398, 2013.
[35] V. M. Suresh, R. Sidhu, P. Karkare, A. Patil, Z. Lei, and A. Basu, "Powering the IoT through embedded machine learning and LoRa," in *Proc. IEEE 4th World Forum Internet Things*, 2018, pp. 349–354.
[36] S. Gutiérrez, I. Martínez, J. Varona, M. Cardona, and R. Espinosa, "Smart mobile LoRa agriculture system based on Internet of Things," in *IEEE 39th Central Amer. Panama Conv.*, 2019, pp. 1–6.
[37] B. P. Rimal, D. P. Van, and M. Maier, "Mobile edge computing empowered fiber-wireless access networks in the 5G era," *IEEE Commun. Mag.*, vol. 55, no. 2, pp. 192–200, Feb. 2017.
[38] T.-Y. Lin et al., "Microsoft coco: Common objects in context," in *Proc. 13th Eur. Conf. Comput. Vis.*, Zurich, Switzerland, Sep. 6–12, 2014, pp. 740–755.
[39] T.-Y. Lin et al., "Microsoft coco: Common objects in context," in *Proc. Eur. Conf. Comput. Vis.*, Cham, Switzerland, 2014, pp. 740–755.
[40] C. Hu et al., "Impact of light and shadow on robustness of deep neural networks," 2023, *arXiv:2305.14165*.
[41] I. Pajač Živković and B. Barić, "The behaviour of codling moth (lepidoptera: Tortricidae) in the croatian apple orchards," *IOBC-WPRS Bull.*, vol. 74, pp. 79–82, Jan. 2012.

Luca Bompani (Student Member, IEEE) received the M.Sc. degree in theoretical physics in 2020 and the second M.Sc. degree in artificial intelligence in 2022 from the University of Bologna, Bologna, Italy, where he is currently working toward the Ph.D. degree in electronic engineering. His research focuses on deploying and optimizing inference of deep neural networks for energy-constrained ultra-low power embedded systems.

Mr. Bompani was the recipient of the Best Paper Award at the IEEE/CVF CVPR'24 Embedded Vision workshop.

Luca Crupi received the M.Sc. degree in computer engineering from Politecnico di Torino, Turin, Italy, in 2022. He is currently working toward the Ph.D. degree with the Dalle Molle Institute for Artificial Intelligence (IDSIA), USI-SUPSI, Lugano, Switzerland. He was part of the XVII of Alta Scuola Politecnica. He was IT lead at PolitOcean, a student team focusing on the development of a fully autonomous underwater vehicle. He has authored or coauthored eight peer-reviewed papers in international conferences and journals. His research focuses on developing AI-based autonomous algorithms for pocket-sized robotic platforms integrating multiple sensor streams to perform sensing with neural network-based techniques.



Daniele Palossi received the Ph.D. degree in information technology and electrical engineering from ETH Zürich, Zürich, Switzerland. He is currently a Senior Researcher with the Dalle Molle Institute for Artificial Intelligence (IDSIA), USI-SUPSI, Lugano, Switzerland, where he leads the nano-robotics research group, and with the Integrated Systems Laboratory (IIS), ETH Zürich, Zürich, Switzerland. He has authored or coauthored more than 45 peer-reviewed papers in international conferences and journals. His research stands at the intersection of artificial intelligence, ultra-low-power embedded systems, and miniaturized robotics.

Dr. Palossi was the recipient of the Swiss National Science Foundation (SNSF) Spark Grant, the 2nd prize at the Design Contest held at the ACM/IEEE ISLPED'19, several Best Paper Awards, and team leader of the winning team of the first "Nanocopter AI Challenge" hosted at the IMAV'22 International Conference.

Olmo Baldoni received the bachelor's degree in automation engineering from the University of Bologna, Bologna, Italy. He is currently working toward the master's degree in computer engineering with a deep passion for machine learning and deep learning, with the University of Modena and Reggio Emilia, Modena, Italy.

Davide Brunelli (Senior Member, IEEE) received the M.S. (cum laude) and Ph.D. degrees in electrical engineering from the University of Bologna, Bologna, Italy, in 2002 and 2007, respectively. He is currently an Associate Professor of electronics with the Department of Industrial Engineering, University of Trento, Trento, Italy. He has authored or coauthored more than 280 research papers in international conferences and journals on ultralow-power embedded systems, energy harvesting, and power management of VLSI circuits. He holds several patents and is annually ranked among the top 2% of scientists according to the "Stanford World Ranking of Scientists" from 2020. His research interests include new techniques of energy scavenging for IoT and embedded systems, the optimization of low-power and low-cost consumer electronics, and the interaction and design issues in embedded personal and wearable devices.

Prof. Brunelli is a Member of several TPC conferences on the Internet of Things and is an Associate Editor for IEEE TRANSACTIONS ON AGRIFOOD ELECTRONICS.

Francesco Conti (Member, IEEE) received the Ph.D. degree in electronic engineering from the University of Bologna, Bologna, Italy, in 2016. He is currently a Tenure-track Assistant Professor with the DEI Department, University of Bologna. From 2016 to 2020, he held a research grant with the University of Bologna and a position as a Post-doctoral Researcher with ETH Zürich, Zürich, Switzerland. His research work has resulted in more than 100 publications in international conferences and journals. His research interets is centered on hardware acceleration in ultra-low power and highly energy-efficient platforms, with a particular focus on system-on-chips for artificial intelligence applications.

Dr. Conti was the recipeint of several awards including 2020 IEEE TRANSACTIONS ON CIRCUITS AND SYSTEMS I: REGULAR PAPERS Darlington Best Paper Award.

Authorized licensed use limited to: Indian Inst of Inform Technology Guwahati. Downloaded on February 16,2026 at 06:55:30 UTC from IEEE Xplore. Restrictions apply.

Manuele Rusci (Member, IEEE) received the Ph.D. degree in electronic engineering from the University of Bologna, Bologna, Italy, in 2018. He is currently holding a MSCA Postdoctoral Fellowship with the Katholieke Universiteit Leuven, Leuven, Belgium, after being Post-doc wih the University of Bologna. His main research interests include low-power AI-powered smart sensors and on-device continual learning.

Luca Benini (Fellow, IEEE) received the Ph.D. degree from Stanford University, Stanford, CA, USA. He is the Chair of digital circuits and systems with ETHZ, Zürich, Switzerland, and is Full Professor with the Università di Bologna, Bologna, Italy. His research interests include energy-efficient parallel computing systems, smart sensing micro-systems, and machine learning hardware.

Prof. Benini is a Fellow of the ACM and a Member of the Academia Europaea and of the Italian Academy of Engineering and Technology.

Authorized licensed use limited to: Indian Inst of Inform Technology Guwahati. Downloaded on February 16,2026 at 06:55:30 UTC from IEEE Xplore. Restrictions apply.